

Programming in C++

Reji J

School of Data Science

IISER TVM



INTRODUCTION TO C++

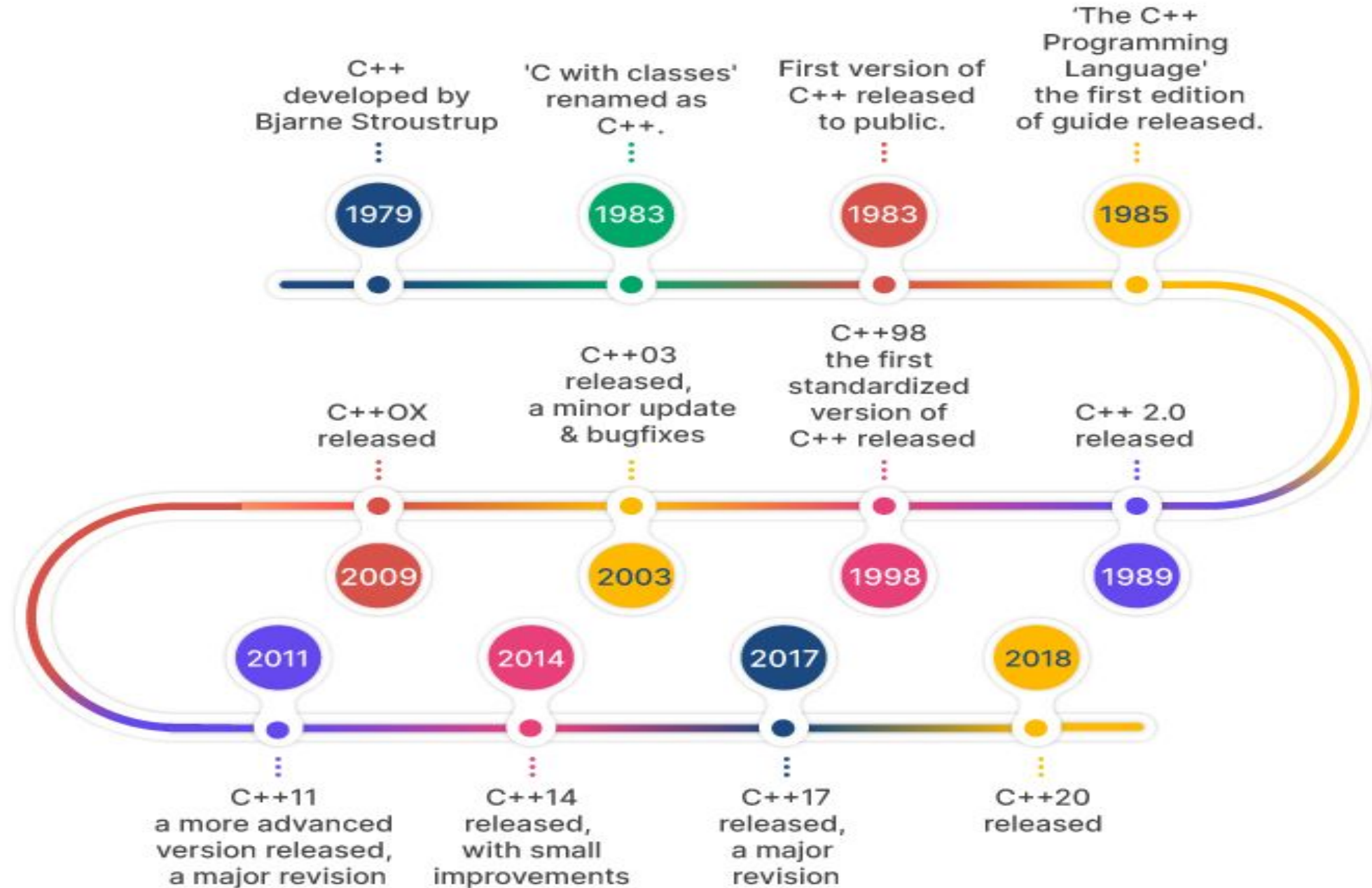
- C++ is a general-purpose, middle-level language that was originally developed by Danish computer scientist, **Bjarne Stroustrup** in 1979 at Bell Laboratories USA.
- A middle-level programming language is a combination of **low-level & high-level programming languages**.
- The C++ language was initially designed as an **extension of the C language** which was widely used in system programming and Unix-based OS. Also, C was developed in 1970 by **Dennis Ritchie** while at Bell Labs (formerly AT&T Bell Labs).
- C++ has become one of the most widely used **programming languages** and has a variety of applications in areas like gaming, robotics, finance, and scientific computing.



- One of the key features of C++ is its ability to support **low-level, system-level programming**, making it suitable for developing operating systems, device drivers, and other system software.
- At the same time, C++ also provides a rich set of libraries and features for **high-level application programming**, making it a popular choice for developing desktop applications, video games, and other complex applications.
- C++ supports **object-oriented programming**, allowing developers to create classes and objects and to define methods and properties for these objects.
- **Templates:** C++ templates allow developers to write generic code that can work with any data type, making it easier to write reusable and flexible code.
- **Standard Template Library (STL):** The STL provides a wide range of containers and algorithms for working with data, making it easier to write efficient and effective code.
- **Exception Handling:** C++ provides robust exception handling capabilities, making it easier to write code that can handle errors and unexpected situations.



HISTORY OF C++



Similarities between C & C++

- Both languages have the same code structure.
- They both have a similar syntax.
- They both have a similar compilation
- Their basic memory model is very close to the hardware.
- Both the languages share the same basic syntax. Also, almost all the operators and keywords of C are present in C++ as well, and they do the same thing.
- As compared to C, C++ has more extended grammar. But the basic grammar here is the same.



Difference between C & C++

- **Input and Output Methods**

- In C, `scanf()` is used for user input, whereas `printf()` is used for outputting data.
- In C++, `std::cin >>` is used for getting user input and `std::cout <<` is used to output data.

- **The Programming Paradigm**

- C is a **procedural oriented language** and its emphasis is on functions.
- Programs are divided into a set of functions and they consist of step-by-step instructions, or commands, to be executed in sequential order, **following a top-down approach**
- C++ is a **Object Oriented Programming language**, which is based on the concept of dividing a program into objects.
- Everything is organized and divided into smaller groups of related parts or *objects*, which are an instance of a *class*, **following a bottom-up approach**.
- This style of programming creates more readable and usable code that is easier to maintain, while also providing better data security.



- **File Extension**

- The file extension for a file that contains C code is `.c`, whereas the file extension for C++ files is `.cpp`

- **The STL**

- C++ offers the STL – Standard Template Library – and C doesn't.

- **Namespaces**

- Namespaces are a feature available in C++ and not C.

- **Exception Handling**

- C doesn't offer a way to handle exceptions in programs which help prevent errors.
 - C++, on the other hand, supports exception handling by introducing `try` and `catch` blocks.



STRUCTURE OF A PROGRAM

Program Code:

```
#include <iostream>
int main()
{
    std::cout << "Hello World!";
    return 0;
}
```

- **Line 1: // my first program in C++**

Indicates comment

- **Line 2: #include <iostream>**

Lines beginning with a hash sign (#) are directives read and interpreted by what is known as the preprocessor.

They are special lines interpreted before the compilation of the program itself begins. In this case, the directive #include <iostream>, instructs the preprocessor to include a section of standard C++ code, known as header iostream, that allows to perform standard input and output operations, such as writing the output of this program (Hello World) to the screen.

- **Line 3: A blank line.**

Blank lines have no effect on a program. They simply improve readability of the code.



Program Code:

```
#include <iostream>
int main()
{
    std::cout << "Hello World!";
}
```

- **Line 4: `int main ()`**

This line initiates the declaration of a function. Essentially, a function is a group of code statements which are given a name: in this case, this gives the name "main" to the group of code statements that follow. Function definition is introduced with a succession of a type (`int`), a name (`main`) and a pair of parentheses (`()`), optionally including parameters.

The function named `main` is a special function in all C++ programs; it is the function called when the program is run. The execution of all C++ programs begins with the `main` function, regardless of where the function is actually located within the code.

- **Lines 5 and 7: `{ and }`**

The open brace (`{`) at line 5 indicates the beginning of main's function definition, and the closing brace (`}`) at line 7, indicates its end. Everything between these braces is the function's body that defines what happens when main is called. All functions use braces to indicate the beginning and end of their definitions.

- **Line 6: `std::cout << "Hello World!";`**

This line is a C++ statement. Statements are executed in the same order that they appear within a function's body.

This statement has three parts: First, `std::cout`, which identifies the standard character output device (usually, this is the computer screen).

Second, the insertion operator (`<<`), which indicates that what follows is inserted into `std::cout`.

Finally, a sentence within quotes ("Hello world!"), is the content inserted into the standard output.



Program Code:

```
#include <iostream>
int main()
{
    std::cout << "Hello World!";
}
```

- Notice that the statement ends with a **semicolon (;)**.
This character marks the end of the statement, just as the period ends a sentence in English. All C++ statements must end with a semicolon character. One of the most common syntax errors in C++ is forgetting to end a statement with a semicolon.
- You may have noticed that not all the lines of this program perform actions when the code is executed. There is a line containing a **comment (beginning with //)**.
- There is a line with a directive for the **preprocessor (beginning with #)**.
- There is a line that defines a **function** (in this case, the **main function**). And, finally, a line with a statements ending with a semicolon (the insertion into cout), which was within the block delimited by the braces ({ }) of the main function.



COMMENTS

- C++ supports two ways of commenting code:

1. `// line comment`
2. `/* block comment */`

- The first of them, known as *line comment*, discards everything from where the pair of slash signs (`//`) are found up to the end of that same line. The second one, known as *block comment*, discards everything between the `/*` characters and the first appearance of the `*/` characters, with the possibility of including multiple lines.

Program Code:

```
/* my second program in C++  
   with more comments */
```

```
#include <iostream>  
int main ()  
{  
    std::cout << "Hello World! ";    // prints Hello World!  
    std::cout << "I'm a C++ program"; // prints I'm a C++ program  
}
```

Output:

Hello World! I'm a C++ program



USING NAMESPACE STD

- `cout` is used instead of `std::cout`.
- Both name the same object: the first one uses its unqualified name (`cout`), while the second qualifies it directly within the namespace `std` (as `std::cout`).
- `cout` is part of the standard library, and all the elements in the standard C++ library are declared within what is called a namespace: the namespace `std`.
- In order to refer to the elements in the `std` namespace a program shall either qualify each and every use of elements of the library (as we have done by prefixing `cout` with `std::`), or introduce visibility of its components. The most typical way to introduce visibility of these components is by means of **using declarations**:

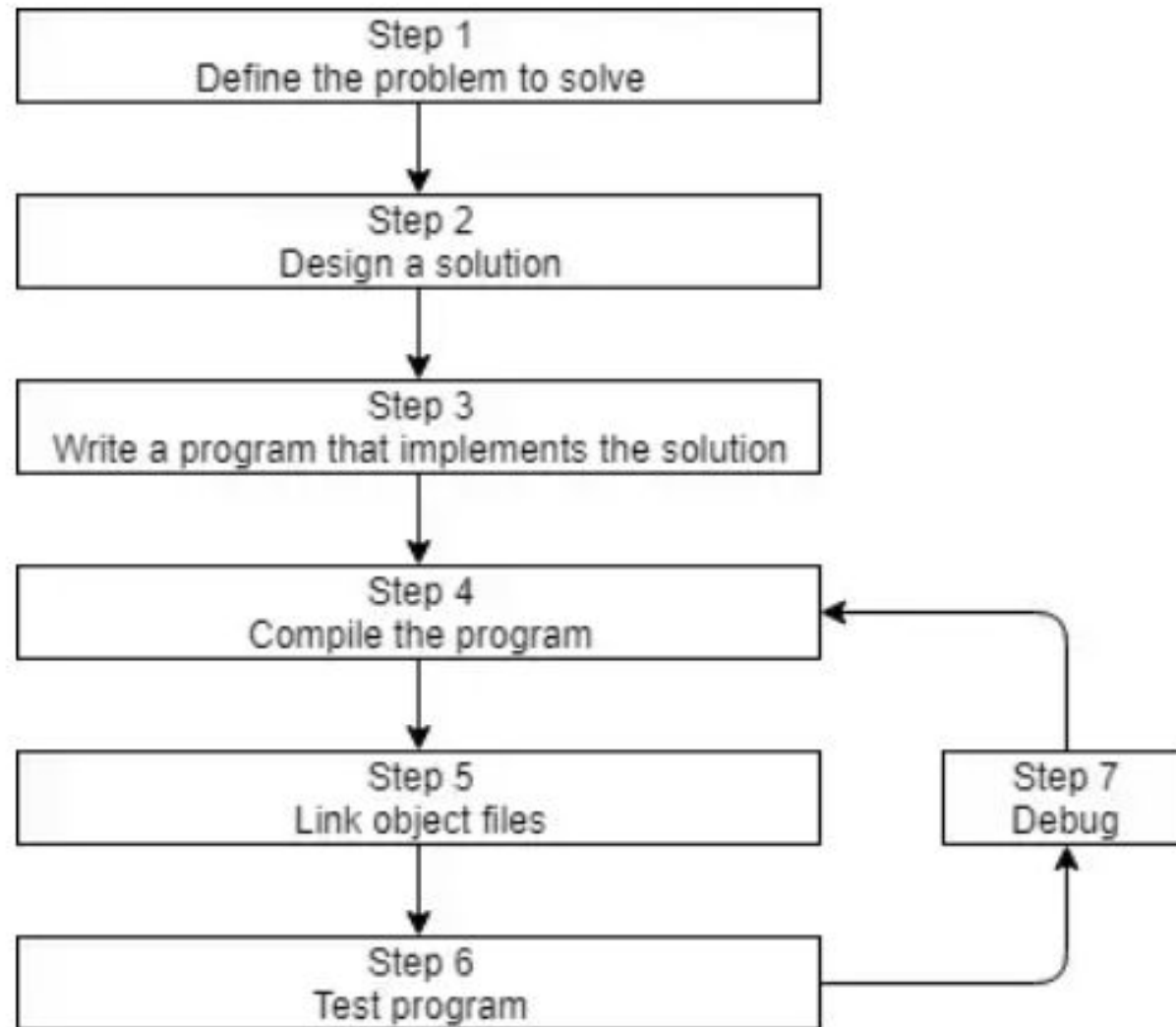
```
using namespace std;
```



Program Code:

```
// my second program in C++  
#include <iostream>  
using namespace std;  
  
int main ()  
{  
    cout << "Hello World! ";  
    cout << "I'm a C++ program";  
    return 0;  
}
```

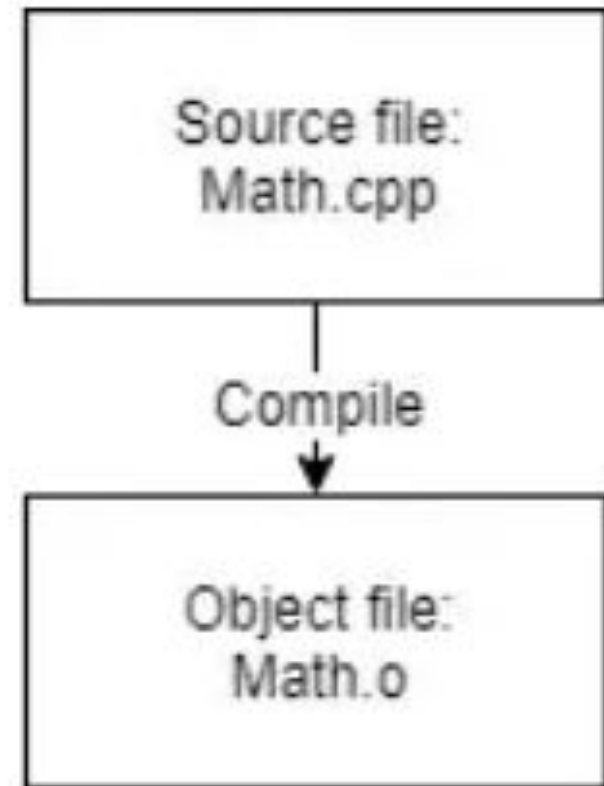
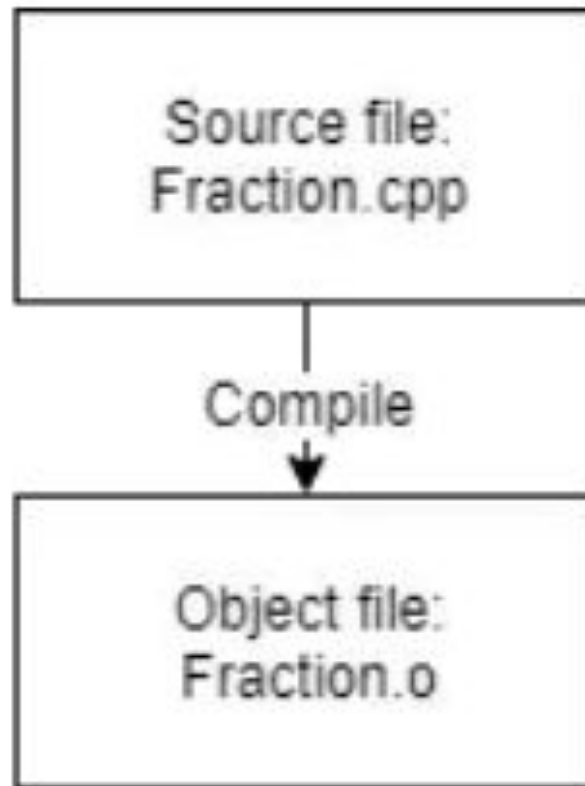
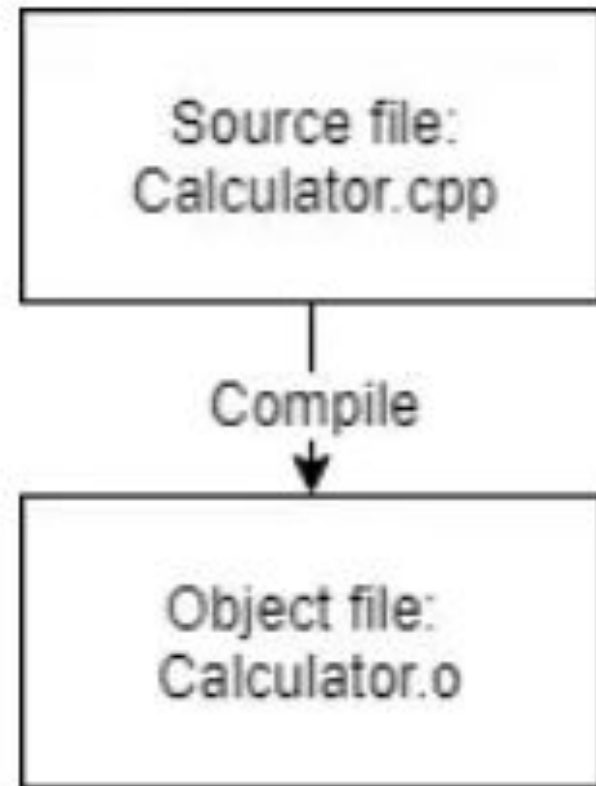




COMPILING YOUR SOURCE CODE

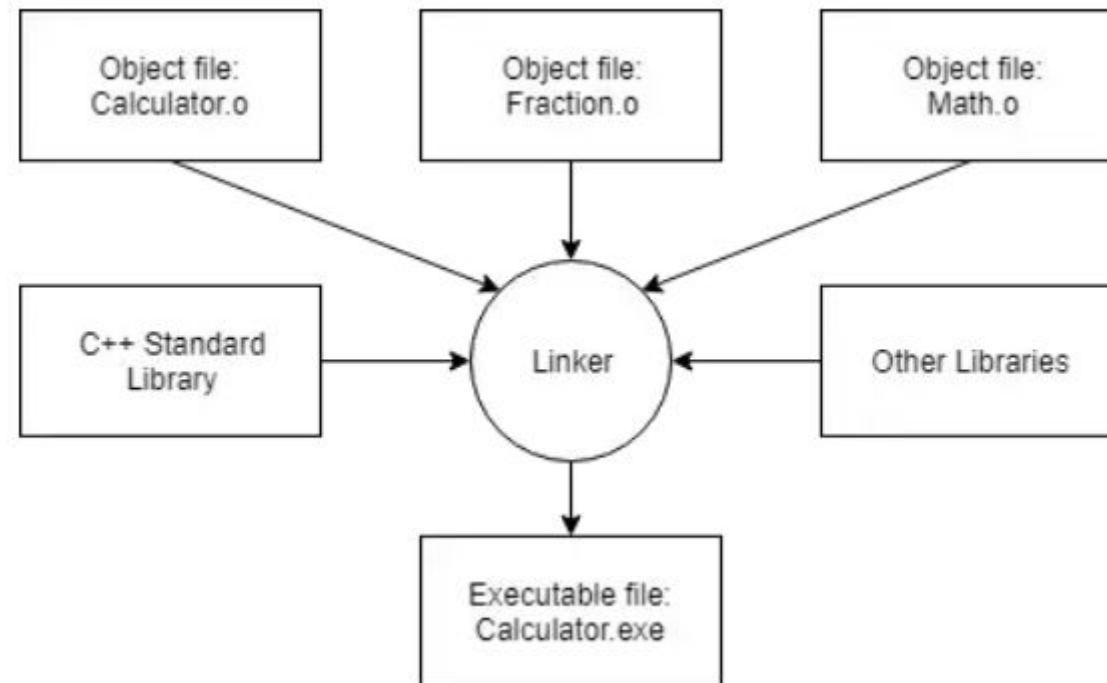
- In order to compile C++ source code files, we use a C++ compiler. The C++ compiler sequentially goes through each source code (.cpp) file in your program and does two important tasks:
- First, it checks your code to make sure it follows the rules of the C++ language. If it does not, the compiler will give you an error (and the corresponding line number) to help pinpoint what needs fixing. The compilation process will also be aborted until the error is fixed.
- Second, it translates your C++ source code into a machine language file called an **object file**. Object files are typically named *name.o* or *name.obj*, where *name* is the same name as the .cpp file it was produced from.





LINKING OBJECT FILES AND LIBRARIES

- After the compiler creates one or more object files, then another program called the **linker** kicks in. The job of the linker is three fold:
- First, to take all the object files generated by the compiler and combine them into a single executable program.



- Second, in addition to being able to link object files, the linker also is capable of linking library files. A **library file** is a collection of precompiled code that has been “packaged up” for reuse in other programs.
- C++ comes with an extensive library called the **C++ Standard Library** (usually shortened to *standard library*) that provides a set of useful capabilities for use in your programs.
- One of the most commonly used parts of the C++ standard library is the *iostream library*, which contains functionality for printing text on a monitor and getting keyboard input from a user.
- Almost every C++ program written utilizes the standard library in some form, so it's very common for the standard library to get linked into your programs. Most linkers will automatically link in the standard library as soon as you use any part of it, so this generally isn't something you need to worry about.



- Third, the linker makes sure all cross-file dependencies are resolved properly. For example, if you define something in one .cpp file, and then use it in another .cpp file, the linker connects the two together. If the linker is unable to connect a reference to something with its definition, you'll get a linker error, and the linking process will abort.
- Once the linker is finished linking all the object files and libraries (assuming all goes well), you will have an executable file that you can then run!



→ **Steps 6 & 7: Testing and Debugging**

- ◆ If your program runs but doesn't work correctly, then it's time for some debugging to figure out what's wrong.

→ **Integrated development environments (IDEs)**

- ◆ Note that steps 3, 4, 5, and 7 all involve software (editor, compiler, linker, debugger). While you can use separate programs for each of these activities, a software package known as an integrated development environment (IDE) bundles and integrates all of these features together.



How to Compile C++ Program in Linux?

1. Open **terminal**

Use the text editor (gedit)

Open file using,

gedit hello.cpp (file name can be anything but it should end with **dot cpp** extension) command.

2. To save the file: **Ctrl + S**. It will save the file.

3. To **compile** the program:

Type,

g++ hello.c

g++ hello.cpp -o hello

4. To **Run** the program:

Type,

./a.out

./hello

5. To Edit the file: Again type **gedit hello.cpp**. Edit your program.

You can Compile and Run C++ Program two ways!

1

Compile the Program

```
g++ hello.cpp
```

- If there are no errors, this will create an **executable file** with the default name **a.out**

Run the Program

```
./a.out
```

2

Compile the Program

```
g++ hello.cpp -o hello
```

- Alternatively, you can specify a custom name for the **executable file**.
- The file will be named **hello** instead of **a.out**.

Run the Program

```
./hello
```